z0p — White Paper

Transparent, Post-Quantum-Oriented Zero-Knowledge Proofs from First Principles

nul0
Future Proofs Tech
futureproofs@proton.me

November 2026

Abstract

We present **z0p**, a complete zero-knowledge proving stack implemented from first principles in safe Rust with no dependencies beyond the compiler and the operating system. Binary fields, the BabyBear prime field, a quartic extension field, BLAKE3-256, Merkle trees, a parallel NTT, FRI, sumcheck, univariate and multilinear polynomial commitments, LogUp lookups, four STARK-style AIR systems, hash-folded IVC, a minimal register-machine AIR, and zero-knowledge masking are owned by one crate under `#![forbid(unsafe_code)]`. The construction is transparent: no trusted setup, no elliptic curves, no pairings. Security reduces to BLAKE3-256, Reed–Solomon proximity testing, and Fiat–Shamir.

Production parameters target a *conjectured* 100-bit query/proof-of-work schedule: rate 1/16, 40 queries, and 20-bit grinding. Algebraic challenges are drawn from the quartic extension $\mathbb{F}_{p^4} = \mathbb{F}_{p^2}[v]/(v^2 - u)$, $\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 - 11)$, giving approximately 124 bits of per-challenge algebraic entropy. On a 16-core reference machine, release profile, `rustc` 1.99.0, a 1,048,576-step Fibonacci AIR proves in 1.73355 s (1.653 μs/step, 604,872 steps/s), verifies in 1.13 ms, and produces a 203,553-byte (198.8 KiB) proof. Verification is essentially independent of trace size: FRI domain points are derived on demand rather than materialized.

The artifact is validated by 277 library tests, 13 security tests with 479 hostile attempts, 10 red-team tests with 37 targeted attacks plus a 500-input garbage storm, 11 hardening tests including exhaustive per-bit AIR decoder fuzzing and 14,000 random mutations across seven encodings, and 5 documentation tests. Every measured forgery is rejected; hostile input fails closed with `Err`, never a panic. The system is explicitly *unaudited*: the contribution is an auditable engineering artifact and a reproducible evidence package, not a claim of production-grade end-to-end soundness.

Contents

1 The problem **2**

2 Design goals and non-goals **2**

3 Threat model and trust base **2**

4 System architecture **3**

5 Cryptographic foundations **4**

6 FRI polynomial commitment **4**

7 Proof systems and commitments **5**

8 AIR systems **7**

9 Zero-knowledge masking **8**

10 Performance evaluation **8**

11 Security evaluation **9**

12 Related work **10**

13 Roadmap and conclusion **10**

A Parameter recipes **10**

B Reproducibility **10**

C Glossary **11**

1 The problem

Practical zero-knowledge infrastructure asks its users to trust a large surface: pairing-friendly curves whose security assumptions are contested, multi-party setup ceremonies whose toxic waste must be destroyed, cryptographic libraries stacked dozens of dependencies deep, and protocol behavior described in papers and slide decks but not reproducible from the code that runs.

At the same time, large-scale quantum computing threatens every deployed pairing- and discrete-log-based proof system. The systems that survive that transition will be hash-based, transparently set up, small enough to audit, and tested hard enough to believe. z0p is an experiment in how small that trusted base can be made while retaining a useful proof toolkit.

2 Design goals and non-goals

Goals. (1) From zero: every cryptographic layer is implemented in the crate; `cargo tree` is one node. (2) Transparent and plausibly post-quantum: BLAKE3 and Reed–Solomon proximity testing; no curves, pairings, or setup. (3) Safe by construction: `#![forbid(unsafe_code)]`, fallible public APIs, canonical fail-closed decoding, hostile parameters re-validated at every entry point. (4) Quantified security: explicit query and proof-of-work budgets. (5) Evidence over claims: adversarial suites, byte-exact proof-size assertions, and a pinned benchmark regression gate. (6) Auditability: one crate, one wire format, one formal specification, and a documented threat model.

Non-goals. z0p is not an audited production system, a general circuit compiler, a zkVM, a private-table lookup protocol, or a side-channel-hardened hardware token. `lookup` and `sumcheck` expose tables in clear; `lookup_pcs` hides columns from the wire but is not zero-knowledge; `basefold`, `gemini`, `lookup_pcs`, `vm`, and `recursion` are prototypes. The approximately 100-bit figure is a query/PoW sizing, not a theorem.

3 Threat model and trust base

Adversary-facing surface. The verifier is the adversary-facing surface. Untrusted inputs are proof and commitment bytes, public parameters (including hand-built `Params` structs whose fields are public), and public statements. The verifier must fail closed with `Err`, never panic,

and never perform unbounded allocation or work. It must derive all challenge-dependent data from the transcript and bind every public input before sampling challenges.

Trust base. The Rust compiler and standard library; BLAKE3-256 as a collision-resistant hash; the Reed–Solomon/FRI proximity heuristic and Fiat–Shamir in the (quantum) random-oracle model; the correctness of the field and NTT implementations, cross-tested against independent Horner/`%` references. No trusted setup, elliptic-curve group, pairing, or discrete-log assumption is used. Shor's algorithm does not apply to the cryptographic core.

Out of scope. Host OS, compiler, CPU microarchitecture, power/EM side channels, physical attacks, denial of service by a prover against itself, and high-value production deployment before an external audit.

4 System architecture

Witness / trace → interpolate and coset LDE (parallel NTT, BabyBear) → composition polynomial (AIR constraints divided by vanishing polynomials, randomized by Fiat–Shamir alphas) → FRI commit (arity-4 folds, Merkle caps, PoW) → proof (caps, final polynomial, query openings, trace openings) → verifier (replay transcript, check paths, folds, degree, identities) → accept/reject.

Layer	Modules	Responsibility
Algebra	`gf2`, `babybear`, `ext4`	Word fields, NTT-friendly prime field, quartic challenge field
Hashing	`hash`, `merkle`	BLAKE3-256, domain separation, caps, index-bound paths
Fiat–Shamir	`transcript`, `rng`	Streaming transcript, challenges, grinding, ChaCha20 CSPRNG
Polynomial core	`ntt`, `mle`, `sumcheck`, `fri`	Coset NTT, multilinear tables, sumcheck, FRI
Commitments	`upcs`, `gemini`, `basefold`	Univariate and multilinear opening prototypes
Lookups	`lookup`, `lookup_pcs`	LogUp in clear and committed-input LogUp
Statements	`air`, `range`, `table`, `fold`, `vm`, `recursion`	STARKs, IVC, execution, recursion fragment
Privacy	`blind`	Vanishing-multiple masking
Wire format	`codec`	Canonical versioned encoding, fail-closed decoding
Parallelism	`par`	Deterministic `std::thread` data parallelism

Table 1: Layer ownership in the crate.

Invariants. The implementation enforces a bilateral rate gate $d + 1 \leq M/2$; transcript binding of dimensions, commitments, public inputs, and PoW nonces; authenticated next rows; committed leaves with index-bound Merkle paths; fail-closed canonical decoding; a non-vacuous FRI final-degree check; and no `unsafe` or runtime dependencies.

5 Cryptographic foundations

Fields. Program words use binary fields $GF(2^8)$, $GF(2^{64})$, $GF(2^{128})$. Polynomial domains use BabyBear

$$p = 2^{31} - 2^{27} + 1 = 2013265921, \qquad p - 1 = 2^{27} \cdot 15,$$

with generator 31. Multiplicative subgroups of order 2^k exist for $k \leq 27$. Elements serialize as four canonical little-endian bytes; multiplication uses Barrett reduction with $\mu = \lfloor 2^{64}/p \rfloor$; inversion is Fermat ($a^{p-2}$, mapping zero to zero); batch inversion uses Montgomery's trick.

All algebraic challenges use the tower

$$\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 - 11), \qquad \mathbb{F}_{p^4} = \mathbb{F}_{p^2}[v]/(v^2 - u),$$

represented as $a + bu + cv + duv$ with four canonical coordinates. 11 is a quadratic non-residue in $\mathbb{F}_p$, and u is a non-square in $\mathbb{F}_{p^2}$. Inversion uses the tower norm: for $x = c + dv$, $x^{-1} = (c - dv)/(c^2 - ud^2)$, so one $\mathbb{F}_{p^2}$ inversion suffices.

Domains and NTT. The trace domain is $H = \{1, \omega, \ldots, \omega^{N-1}\}$; the evaluation domain is the coset $D = \gamma \cdot \langle \Omega \rangle$ of size $M = N\beta$, with coset shift $\gamma = 31$ and $M \leq 2^{27}$. The vanishing polynomial is $Z_H(x) = x^N - 1$. The NTT is a table-driven Cooley–Tukey transform with bit-reversal, deterministic parallel stages, and inverse scaling folded into the last stage. Reed–Solomon extension interpolates and evaluates on the coset.

BLAKE3 and Merkle trees. Unkeyed BLAKE3-256 is implemented from the specification: 7-round ARX compression, the standard message permutation, flags `CHUNK_START`, `CHUNK_END`, `PARENT`, `ROOT`, 1024-byte chunks, and Bao-style merge-while-even stacking. Column trees use

$$\text{leaf}(d) = H(0x00 \| d), \qquad \text{node}(l, r) = H(0x01 \| l \| r).$$

FRI layers commit one 64-byte leaf per quad (four 16-byte $\mathbb{F}_{p^4}$ values); AIR trace layers commit one 64-byte leaf per eight joint rows. Caps expose an upper tree layer of 2^h digests, and every path direction bit is bound to the claimed leaf index. Cap height is capped at 8.

Transcript. The transcript is append-only and length-prefix framed. Each absorb writes (label_len, label, data_len, data) into a streaming BLAKE3 state. A challenge clones that state, appends (label_len, label, counter), finalizes, and feeds the output back. This is byte-identical to hashing the full concatenation but independent of history length. $\mathbb{F}_p$ challenges reduce a 64-bit word modulo p; query indices mask low bits for power-of-two domains; $\mathbb{F}_{p^4}$ challenges reduce four coordinates.

Proof of work. `grind(label,d)` finds the smallest nonce with at least d leading zero bits under the history commitment, using a banded parallel search identical to a sequential scan. Protocol parameters cap grinding at 32 bits; `grind` fails closed above 32. `verify_grind` checks the nonce and absorbs it only on success. Queries are sampled after the nonce is absorbed.

6 FRI polynomial commitment

A FRI parameter set is (β, Q, F, g, h) with $\beta \geq 2$, $Q \geq 1$, $2 \leq F \leq 1024$, $g \leq 32$, $h \leq 8$, $F < M$, and M/F a power of four. `commit` maps n coefficients to $M = n\beta$ coset evaluations with a shifted NTT. `commit_evals` accepts already-expanded LDE values with a declared degree bound $d < M$. Both paths validate parameters before the encoding NTT, reject empty or non-power-of-two lengths, enforce $M \leq 2^{27}$, and bind (M, n, d) under `fri-dims` before any challenge. The rate gate $d + 1 \leq M/2$ is enforced at prove and verify.

Arity-4 folding. Layers are bit-reversed. Each adjacent quad sits at $(x, -x, j_0 x, -j_0 x)$ for the fixed fourth root j_0, sharing x^4. Writing $P(X) = A + BX + CX^2 + DX^3$ in $Y = X^4$, the fold is

$$\text{fold}(e_0, e_1, e_2, e_3) = A + r_0 B + r_1 C + r_2 D,$$

with

$$\begin{aligned} A &= (e_0 + e_1 + e_2 + e_3)/4, \\ B &= ((e_0 - e_1) - j_0(e_2 - e_3))/(4x), \\ C &= ((e_0 + e_1) - (e_2 + e_3))/(4x^2), \\ D &= ((e_0 - e_1) + j_0(e_2 - e_3))/(4x^3), \end{aligned}$$

and $(r_0, r_1, r_2) \in \mathbb{F}_{p^4}^3$. Point inverses are base-field values and are batch-inverted once per layer. Each fold quarters the degree.

Proof and verification. A proof carries the final polynomial in the clear, one opening per query (four $\mathbb{F}_{p^4}$ values per folded quad plus the middle Merkle path segment), and a PoW nonce. The verifier validates dimensions, cap shapes, fold count, and the rate gate; absorbs dimensions, cap digests, and the final polynomial; checks the PoW; verifies the final layer by divided differences against residual $d \gg 2f$; recomputes the final cap from `final_poly`; and checks every query path and fold relation. A one-point final layer is rejected because it would make the degree check vacuous; a fold-free instance is rejected because it has no query openings binding `final_poly` to the commitment.

Conjectured soundness. Under the Johnson-radius heuristic, a word at distance δ from the code survives one query with probability at most $1 - \delta$, where $\delta \approx 1 - \sqrt{\rho}$.

Shape	Rate ρ	Queries Q	Grinding	Conjectured query+PoW
Bench	1/8	12	0	≈ 18 bits
Production	1/16	40	20	≈ 100 bits $(80 + 20)$

Table 2: Conjectured FRI query/PoW schedule. Algebraic error is separate.

These counts exclude finite-field algebraic error. FRI folds, STARK compositions, UPCs/Gemini points and betas, sumcheck rounds, and lookup identities use $\mathbb{F}_{p^4}$, so a degree-d identity collides with probability about d/p^4 per challenge ($\approx$ 124 bits). This remains a query/PoW formula, not an end-to-end theorem.

7 Proof systems and commitments

Sumcheck. A multilinear polynomial with n variables stores 2^n evaluations in MSB-first order. The equality polynomial is

$$\text{eq}(r, x) = \prod_i (r_i x_i + (1 - r_i)(1 - x_i)).$$

A summand is a sum of terms $f(x) = \sum_k c_k \prod_{j \in \text{factors}_k} \text{mle}_j(x)$ with $c_k \in \mathbb{F}_{p^4}$. Sumcheck proves $S = \sum_{x \in \{0,1\}^n} f(x)$ by sending round polynomials of degree at most the maximum factor count. Verification checks $g_i(0) + g_i(1) = g_{i-1}(r_{i-1})$, samples r_i, and finally checks the oracle value.

UPCs. `upcs` commits a univariate polynomial as a Merkle-committed Reed–Solomon LDE. To prove $g(\beta) = y$, the prover commits $q = (g - y)/(X - \beta)$ and proves both g and q low-degree with FRI; the verifier checks $(X - \beta)q = g - y$ at the quotient's query points. Multi-point openings use I interpolating the claims and $Z = \prod_j (X - p_j)$, with $q = (g - I)/Z$ and $g = I + Zq$. When the points determine the polynomial, the quotient is zero and the verifier checks g against the interpolant directly.

Gemini. Gemini commits the monomial-coefficient encoding $g_0(X) = \sum_b c_b X^{\text{index}(b)}$ and proves $f(r) = y$ by folding one variable at a time:

$$g_{i-1}(X) = A_i(X^2) + XB_i(X^2), \qquad g_i(Y) = A_i(Y) + r_i B_i(Y).$$

Each committed layer is opened once with a multi-point UPCs proof at all points its relations use. Layer commitments are absorbed before the fold betas. Proof size is linear, not constant.

BaseFold prototype. BaseFold commits the coefficient encoding with FRI and reduces $f(r) = y$ to a coefficient-oracle evaluation $c(\rho)$ with sumcheck. The prover folds the LDE with the Möbius/eq rule $(1 - \rho_i)E(X) + \rho_i O(X)$ rather than the coefficient-encoding rule $E(X) + \rho_i O(X)$, so the final FRI constant is exactly $c(\rho)$. The verifier recomputes

$$w(\rho) = \prod_i (1 - \rho_i + \rho_i r_i)$$

and checks $f(r) = c(\rho)w(\rho)$. Proofs carry one root, $n - 1$ layer roots, sumcheck rounds, and one pair plus one Merkle path per layer per query. Optional caps store a digest frontier per layer and hash it back to the layer root. Batched multi-point openings use the multilinear basis table and are capped at 1024 points. The prototype is not wired into the AIR/STARK stack and has no end-to-end soundness proof; it backs the committed lookup.

LogUp lookups. For witness w and table t padded to powers of two, the prover computes first-index multiplicities m, samples β avoiding poles, and forms

$$q_i^w = \frac{1}{\beta + w_i}, \qquad q_j^t = \frac{m_j}{\beta + t_j}.$$

Four sumchecks establish the rational identity $\sum_i q_i^w = \sum_j q_j^t$ and the read-checks $q_i^w(\beta + w_i) = 1$, $q_j^t(\beta + t_j) = m_j$. The verifier has witness and table in clear and recomputes canonical multiplicities exactly; fractional multiplicities are rejected. The API is sound for membership but not hiding.

`lookup_pcs` commits witness, table, multiplicities, and both quotient columns with BaseFold. The verifier sees five commitments and eight openings. The read-checks use the committed columns directly:

$$\text{eq} \cdot \big(q \cdot (\beta + v) - \text{target}\big) = 0,$$

expanded as $\text{eq}qv + \beta\text{eq}q - \text{eq}\,\text{target}$. A fresh γ folds the total multiplicity weight into the sum-t claim: $\sum(q^t + \gamma m) = \sum q^w + \gamma W$. The verifier rejects $\gamma = 0$ and opens every final sumcheck value against its commitment. At 1,024-in-1,024 with $B16Q40G20C4$, the prototype measures 273.01 ms prove, 5.77 ms verify, and 642,443 bytes. It is committed-input but not zero-knowledge and does not constrain columns to the base field.

8 AIR systems

Fibonacci AIR. Trace $A[0] = a_0$, $B[0] = b_0$, $A[i+1] = B[i]$, $B[i+1] = A[i] + B[i]$, claim $B[N-1] = b_{\text{last}}$. With $w_{\text{last}} = \omega^{N-1}$ and $Z_{\text{trans}}(x) = (x^N - 1)/(x - w_{\text{last}})$:

$$C(x) = \alpha_0 \frac{a' - b}{Z_{\text{trans}}(x)} + \alpha_1 \frac{b' - (a+b)}{Z_{\text{trans}}(x)} + \alpha_2 \frac{a - a_0}{x - 1} + \alpha_3 \frac{b - b_0}{x - 1} + \alpha_4 \frac{b - b_{\text{last}}}{x - w_{\text{last}}}.$$

Five `fib/alpha` challenges; $\deg C < N$ iff the trace is valid. Dividing, not multiplying, by Z_{trans} is load-bearing. Blowup is at least 4. Next-row values are the same committed columns at the ω-shifted LDE index, authenticated by a second Merkle path.

Range check AIR. Value column V plus n bit columns ($1 \le n \le 24$), with challenges α_{decomp}, $\{\alpha_j\}$, α_{bound}:

$$C = \alpha_{\text{decomp}} \frac{V - \sum_j 2^j C_j}{Z_H} + \sum_j \alpha_j \frac{C_j(C_j - 1)}{Z_H} + \alpha_{\text{bound}} \frac{V - \text{value}}{x - 1}.$$

Booleanity doubles the masked degree; masked callers should size blowup and queries for a rate near $2/\beta$. The verifier caps the declared bound at the largest honest masked degree.

Table membership AIR.

$$C(x) = \alpha_{\text{member}} \frac{\prod_{t \in T}(v - t)}{Z_H(x)} + \alpha_{\text{bound}} \frac{v - \text{value}}{x - 1}.$$

The honest composition bound is $|T|(n + k - 1) - n$ for n rows and k maskers. The tiny-table `for_security` tuner floors blowup at 4 so FRI always has a valid final layer.

Hash-folded IVC. Segments of `seg_steps` Fibonacci rows are committed independently with explicit in/out claims. The verifier checks chain links, then verifies one shared FRI proof over $V = \sum_s \gamma_s C_s$ with per-step challenges. A far-from-code segment keeps the combination far with overwhelming probability over the gammas. The statement is existential over the number of segments; callers requiring an exact step count must check `proof.steps.len()`. The accumulator retains each segment's LDE, Merkle tree, and composition until finalization.

Register-machine execution. A public two-register straight-line program runs from a private initial state to a public output. Opcodes are `Add`, `Mul`, `Swap`, `SetA`, `SetB`, and `Halt`; the last row is the public output and has no outgoing transition. Public one-hot selectors and immediate polynomials drive transition numerators T, U:

$$C(x) = \alpha_0 \frac{T(x)}{Z_{\text{trans}}(x)} + \alpha_1 \frac{U(x)}{Z_{\text{trans}}(x)} + \alpha_2 \frac{A(x) - a_{\text{last}}}{x - w_{\text{last}}} + \alpha_3 \frac{B(x) - b_{\text{last}}}{x - w_{\text{last}}}.$$

Plain composition degree is below $2n - 1$, so blowup is at least 8. Masked columns have degree padded $- 1$; the `Mul` transition is quadratic, so the prover declares $2(n + n_{\text{rand}} - 1)$ and the verifier accepts up to $2 \cdot \text{padded} - 2$. Each query reveals the current and shifted rows, so the mask budget is $2Q$. The program and output remain public; there is no control flow or memory.

Recursion fragment. `recursion` proves one FRI query's fold chain as a small STARK. Public inputs are the outer `lde_len`, leaf, per-depth challenges, the layer-0 quad, and the output; the witness is the sequence of quads and folded outputs. Constraints cover the fold identity, the selected next-layer element, the initial quad, and the final output. The fragment does not yet verify BLAKE3 paths, the final degree check, or transcript derivation inside the AIR.

9 Zero-knowledge masking

Masked proving maps a column T of degree $< n$ to

$$M(x) = T(x) + Z_H(x)R(x),$$

where R has n_{rand} uniform coefficients. Z_H vanishes on every base row, so trace values and AIR quotient identities are preserved. Off-domain, the mask term is nonzero. Evaluations of R at k distinct points are jointly uniform when $n_{\text{rand}} \geq k$ (Vandermonde), giving perfect hiding of query responses.

Protocol	Revealed values per column per query	Minimum n_{rand}
`range`, `table`	one point	Q
`vm`	current and shifted rows	$2Q$
`air`	eight-row leaf and shifted leaf	$16Q$

Table 3: Masking budgets for the values actually revealed.

The composition degree grows accordingly; callers must size `final_size` for the masked LDE. Verification is unchanged, and masked false statements still fail. This is query-response hiding, not full program privacy: public inputs, Merkle roots, and composition commitments remain visible. `lookup`, `sumcheck`, `upcs`, `gemini`, and `basefold` are not hiding.

10 Performance evaluation

Method. `benches/speed_size.rs`, release profile, median of runs after one warm-up, 16 cores / 31 GB, `rustc` 1.99.0, measured 2026-11-05. Every benchmark verifies its proof once outside the timed region; byte counters assert `to_bytes().len()` exactly. Proof sizes are deterministic. The regression gate allows 3× timing drift and requires byte-exact sizes.

Workload (production query/PoW)	Prove	Verify	Size
AIR 1,024 steps	37.05 ms	0.48 ms	68,573 B
AIR 8,192 steps	49.81 ms	0.59 ms	101,233 B
AIR 65,536 steps	115.49 ms	0.90 ms	141,641 B
AIR **1,048,576** steps	**1,733.55 ms**	**1.13 ms**	**203,553 B**
FRI poly-1,024	21.50 ms	0.31 ms	43,174 B

Table 4: Production AIR/FRI measurements, 2026-11-05.

The 1M-step row is 1.653 μs/step and 604,872 steps/s. Proof size grows logarithmically; verification stays in the low-millisecond range from 1,024 to 1,048,576 steps. On the optional nightly SIMD hash path, a representative run measured 1,693.58 ms for the same row (approximately 2.3% faster); the default scalar path remains the supported stable build.

Cap height 4 reduces the BaseFold proof by 37.3%. Against Gemini, uncapped BaseFold is approximately 14.0× faster to prove, 8.4× faster to verify, and 7.0× smaller; capped is approximately 13.9×, 11.7×, and 11.1×. The BaseFold and committed-lookup rows are prototypes; the VM row is the plain path.

Construction	Prove	Verify	Size
BaseFold-style, 1,024 coefficients	22.65 ms	0.88 ms	127,658 B
BaseFold-style, cap height 4	22.91 ms	0.63 ms	80,018 B
Gemini v1, 1,024 coefficients	317.96 ms	7.36 ms	887,872 B
`lookup_pcs`, 1,024-in-1,024, C4	273.01 ms	5.77 ms	642,443 B
`vm`, 64 steps, $B16Q12C2$	0.69 ms	0.18 ms	14,933 B

Table 5: Multilinear PCS, committed lookup, and execution measurements.

Primitives. BLAKE3 1 MiB: 1.17 ms (856.5 MB/s). BabyBear multiplication: 6.59 ms/1M (151.8 Mops/s). $GF(2^{64})$ multiplication: 272.84 ms/1M (3.7 Mops/s). NTT 2^{14}: 0.32 ms. 64k-leaf Merkle tree: 3.24 ms.

11 Security evaluation

Assumptions. BLAKE3 collision/preimage resistance; FRI soundness under the Johnson-radius heuristic; Fiat–Shamir in the (quantum) random-oracle model; standard Reed–Solomon list-decoding bounds. No discrete-log, pairing, or setup assumptions: the construction is Shor-inert by design.

Enforced invariants. Bilateral rate gate; verifier-derived domains and query indices; non-vacuous final divided-difference check; cap/path shape pins before hashing; direction bits bound to claimed indices; `fri-dims` dimension binding; canonical-only decoding; PoW bound to history with queries sampled after; committed read-checks in `lookup_pcs`; correct masked composition bounds in VM; and next-row openings authenticated against the same trace cap.

Adversarial evidence. The current tree passes 277 library tests, 13 security tests with 479 hostile attempts, 10 red-team tests with 37 targeted attacks plus a 500-input garbage storm, 11 hardening tests including exhaustive per-bit AIR decoder fuzzing and 14,000 random mutations across seven encodings, and 5 documentation tests: 316 tests total, plus the coverage-guided `fuzz/decode` target. Every measured forgery is rejected; hostile input fails closed with `Err`, never a panic.

Closed findings. Internal adversarial review has closed Merkle path index substitution across all verifiers and hand-rolled FRI/BaseFold walks; one-point FRI final layers and fold-free FRI; final cap not bound to `final_poly`; `lookup_pcs` read-checks using uncommitted shifted MLEs; VM masked `Mul` degree under-declaration; AIR/VM ZK mask budgets undercounting revealed values; batched BaseFold openings using the equality table instead of the multilinear basis table; and prover allocation ordering, cap-height/roundtrip limits, recursion domain bounds, checked buffer arithmetic, and transcript/grinding hardening.

Residual risk. The system is unaudited. The query/PoW figures are conjectured. The small-table AIR scales with table size. `lookup_pcs` is not zero-knowledge and does not constrain columns to the base field. BaseFold, Gemini, VM, and recursion are prototypes. Side channels in the host OS, compiler, or CPU are out of scope. Do not deploy for high-value statements before an external audit.

12 Related work

z0p follows the STARK/FRI lineage: Ben-Sasson et al.'s STARKs and FRI, DEEP-FRI, and BCS-style Fiat–Shamir. Production systems such as Plonky2 and RISC Zero share the BabyBear-plus-FRI engineering core but retain larger dependency footprints. z0p differs by taking dependency-freedom and auditable minimalism as hard constraints, and by publishing adversarial test evidence alongside performance claims. Lookups follow the LogUp logarithmic-derivative design. The hash is BLAKE3; Fiat–Shamir is classical.

13 Roadmap and conclusion

The evidence package is ready for external review. The next steps are a hiding PCS and sumcheck for private tables; batched BaseFold openings to replace the eight separate `lookup_pcs` openings; self-proving recursion with BLAKE3 path checks, final-degree checks, and transcript derivation inside the AIR; control flow, memory, instruction privacy, and a general circuit compiler for the zkVM direction; and broader SIMD coverage.

z0p demonstrates that a useful, transparent STARK stack can be implemented from first principles in safe Rust with zero dependencies and a small, auditable trusted base. The current artifact is deliberately positioned as pre-audit research software: the engineering, adversarial evidence, formal specification, and measured evaluation are complete enough for external review, while the soundness figures remain conjectured and the prototype components remain explicitly labeled.

A Parameter recipes

Constructor	Key validation
`fri::Params::new`	blowup ≥ 2, queries ≥ 1, final $2 \dots 1024$
`air::Params::new`	N power of two ≥ 2, blowup ≥ 4, $M \leq 2^{27}$
`range::Params::new`	bits $1 \dots 24$, blowup ≥ 2
`table::Params::new`	table checked at prove/verify, blowup ≥ 2
`fold::Params::new`	segment ≥ 2, blowup ≥ 4
`vm::Params::new`	blowup ≥ 8, program $\leq N - 1$
`recursion::Params::new`	folds $1 \dots 16$, outer LDE $\leq 2^{27}$
`basefold::Params::new`	blowup ≥ 2, cap ≤ 8, vars ≤ 16
`gemini::Params::new`	blowup ≥ 8, vars ≤ 20
`upcs`	points ≤ 16, blowup ≥ 2, LDE $\leq 2^{27}$

B Reproducibility

1. `cargo fmt -check`
2. `python3 tools/check_links.py`
3. `RUSTDOCFLAGS="-D warnings" cargo doc -no-deps`
4. `cargo clippy -all-targets - -D warnings`
5. `cargo test` and `cargo test -release`
6. `cargo bench` (24 rows)

7. `python3 tools/check.py` (full gate)

The full gate pins all 24 benchmark rows: proof sizes byte-for-byte, times within 3× of `tools/baseline.json`. See `AUDIT.md`, `docs/benchmarks.md`, and `docs/security-tests.md` for the evidence package.

C Glossary

Term	Meaning
AIR	Algebraic Intermediate Representation
FRI	Fast Reed–Solomon IOP of Proximity
LDE	Low-degree extension
MLE	Multilinear extension
PCS	Polynomial commitment scheme
PoW	Proof of work / Fiat–Shamir grinding
STARK	Scalable Transparent Argument of Knowledge
UPCs	Univariate polynomial commitment scheme
ZK	Zero knowledge

References

[1] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018.

[2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *ICALP 2018*.

[3] E. Ben-Sasson, D. Bentov, Y. Horesh, and M. Riabzev. DEEP-FRI: Sampling outside the box improves soundness. Cryptology ePrint Archive, Report 2019/336, 2019.

[4] E. Ben-Sasson, A. Chiesa, and N. Spooner. Interactive oracle proofs. In *TCC 2016*.

[5] A. Fiat and A. Shamir. How to prove yourself. In *CRYPTO 1986*.

[6] J.-P. Aumasson, S. Neves, J. O'Connor, and Z. Wilcox-O'Hearn. BLAKE3: one function, fast everywhere. https://github.com/BLAKE3-team/BLAKE3, 2020.

[7] U. Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022.